

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ОТЧЕТ**

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

**«Запросы на выборку и модификацию данных. Представления.
РАБОТА С ИНДЕКСАМИ»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Лукис Вадим Вадимович

Факультет прикладной информатики

Группа К3242

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

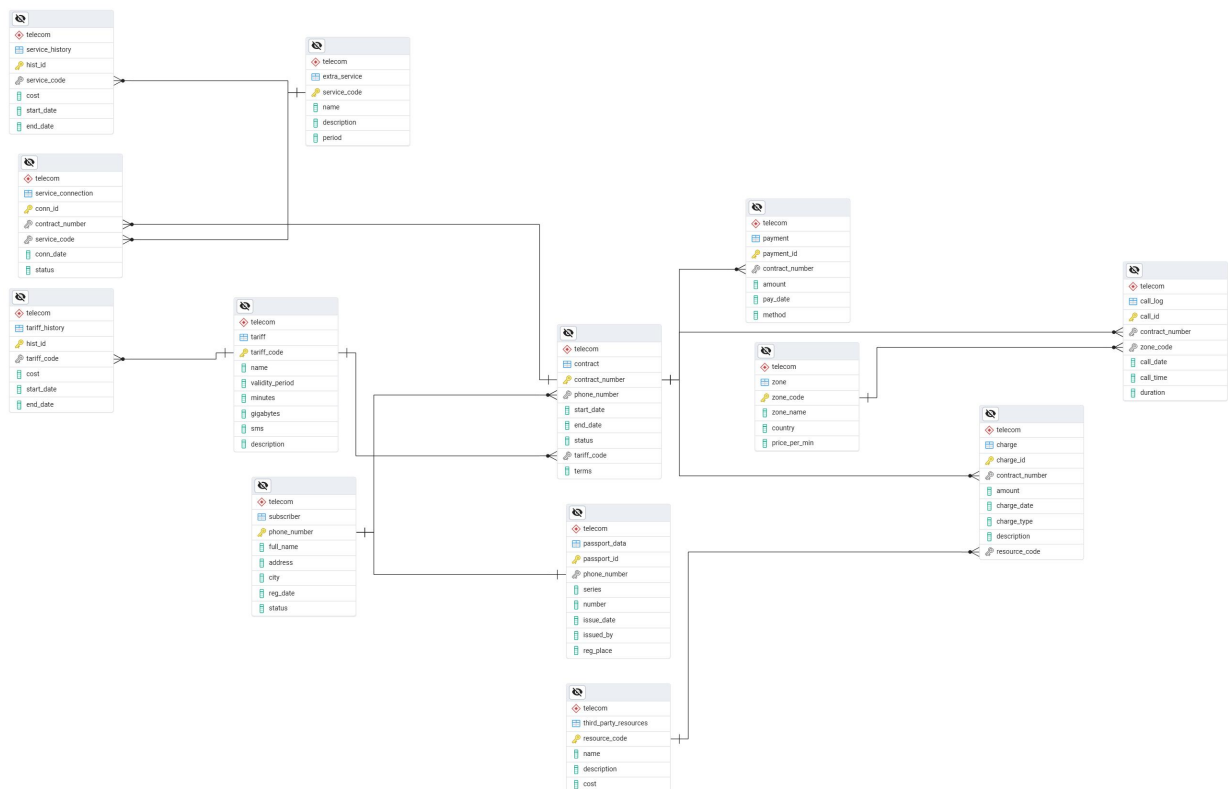
Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема базы данных



Выполнение

Наименование БД: TelecomDB

Вариант: 17

Для проверки работы запросов с временными условиями (например, «за последнюю неделю») и фильтрацией по времени суток, в базу данных были добавлены еще дополнительные тестовые данные (после добавленных еще во время ЛР3).

Дата генерации: 01.05.2026.

Добавлено: новые абоненты, договоры, паспортные данные, а также звонки (включая ночные звонки, звонки за последнюю неделю и междугородние вызовы внутри РФ).

Запросы к базе данных

Часть 2:

1. Вывести суммарное время переговоров каждого абонента за заданный период.

```
SELECT s.phone_number, s.full_name,  
       SUM(cl.duration) as total_duration_minutes  
FROM telecom.subscriber s  
JOIN telecom.contract c ON s.phone_number = c.phone_number  
JOIN telecom.call_log cl ON c.contract_number = cl.contract_number  
WHERE cl.call_date BETWEEN '2026-04-01' AND '2026-04-30'  
GROUP BY s.phone_number, s.full_name  
ORDER BY total_duration_minutes DESC;
```

Query Query History

```
1  SELECT s.phone_number, s.full_name,  
2      SUM(cl.duration) as total_duration_minutes  
3  FROM telecom.subscriber s  
4  JOIN telecom.contract c ON s.phone_number = c.phone_number  
5  JOIN telecom.call_log cl ON c.contract_number = cl.contract_number  
6  WHERE cl.call_date BETWEEN '2026-04-01' AND '2026-04-30'  
7  GROUP BY s.phone_number, s.full_name  
8  ORDER BY total_duration_minutes DESC;
```

Data Output Messages Notifications



	phone_number [PK] character varying (12)	full_name character varying (100)	total_duration_minutes bigint
1	+79001112233	Иванов Иван Иванович	3360
2	+79007778899	Сидорова Анна Сергеевна	2100
3	+79004445566	Петров Петр Петрович	1440
4	+79005556677	Ночной Павел Петрович	1060
5	+79001234567	Смирнов Алексей Владимиров...	750
6	+79009876543	Козлова Мария Игоревна	600

2. Найти среднюю продолжительность разговора абонента АТС.

```
SELECT AVG(duration) as avg_call_duration  
FROM telecom.call_log;
```

The screenshot shows a SQL query editor with two tabs: "Query" and "Query History". The "Query" tab is active, displaying a SQL query:

```
1 SELECT AVG(duration) as avg_call_duration
2 FROM telecom.call_log;
```

Below the query editor is a toolbar with icons for Data Output, Messages, and Notifications. The "Data Output" tab is active, showing a table with the following data:

	avg_call_duration	
1	283.33333333333333	

3. Вывести количество междугородных переговоров каждого абонента.
(Я специально посмотрел, что значит междугородные: **междугородная** связь - телефонная связь между двумя различными регионами в пределах одной страны.)
Здесь я сделал для России.

```
SELECT s.phone_number, s.full_name,  
       COUNT(cl.call_id) as intercity_calls_count  
FROM telecom.subscriber s  
JOIN telecom.contract c ON s.phone_number = c.phone_number  
JOIN telecom.call_log cl ON c.contract_number = cl.contract_number  
JOIN telecom.zone z ON cl.zone_code = z.zone_code  
WHERE z.country = 'Россия'  
      AND z.zone_name NOT LIKE '%' || s.city || '%'  
GROUP BY s.phone_number, s.full_name;
```

Query
Query History

```

1 SELECT s.phone_number, s.full_name,
2       COUNT(cl.call_id) as intercity_calls_count
3 FROM telecom.subscriber s
4 JOIN telecom.contract c ON s.phone_number = c.phone_number
5 JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
6 JOIN telecom.zone z ON cl.zone_code = z.zone_code
7 WHERE z.country = 'Россия'
8       AND z.zone_name NOT LIKE '%' || s.city || '%'
9 GROUP BY s.phone_number, s.full_name;

```

Data Output
Messages
Notifications

	phone_number [PK] character varying (12)	full_name character varying (100)	intercity_calls_count bigint
1	+79001112233	Иванов Иван Иванович	6
2	+79001234567	Смирнов Алексей Вла...	1
3	+79004445566	Петров Петр Петрович	6
4	+79005556677	Ночной Павел Петрович	2
5	+79007778899	Сидорова Анна Серге...	8
6	+79009876543	Козлова Мария Игоре...	1

4. Вывести список абонентов, продолжительность разговоров которых за прошедшую неделю была максимальной.
Я ограничил тремя.

```

SELECT s.phone_number, s.full_name,
       SUM(cl.duration) as total_duration
FROM telecom.subscriber s
JOIN telecom.contract c ON s.phone_number = c.phone_number
JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
WHERE cl.call_date >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY s.phone_number, s.full_name
ORDER BY total_duration DESC
LIMIT 3;

```

Query Query History

```

1 SELECT s.phone_number, s.full_name,
2       SUM(cl.duration) as total_duration
3 FROM telecom.subscriber s
4 JOIN telecom.contract c ON s.phone_number = c.phone_number
5 JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
6 WHERE cl.call_date >= CURRENT_DATE - INTERVAL '7 days'
7 GROUP BY s.phone_number, s.full_name
8 ORDER BY total_duration DESC
9 LIMIT 3;

```

Data Output Messages Notifications

	phone_number [PK] character varying (12)	full_name character varying (100)	total_duration bigint
1	+79001112233	Иванов Иван Иванович	1500
2	+79007778899	Сидорова Анна Сергее...	960
3	+79004445566	Петров Петр Петрович	600

5. Сколько звонков было сделано в каждый из городов России.

```

SELECT z.zone_name, z.country,
       COUNT(cl.call_id) as calls_count
FROM telecom.call_log cl
JOIN telecom.zone z ON cl.zone_code = z.zone_code
WHERE z.country = 'Россия'
GROUP BY z.zone_name, z.country
ORDER BY calls_count DESC;

```

Query Query History

```

1 SELECT z.zone_name, z.country,
2       COUNT(cl.call_id) as calls_count
3 FROM telecom.call_log cl
4 JOIN telecom.zone z ON cl.zone_code = z.zone_code
5 WHERE z.country = 'Россия'
6 GROUP BY z.zone_name, z.country
7 ORDER BY calls_count DESC;

```

Data Output Messages Notifications

	zone_name character varying (50)	country character	calls_count bigint
1	Москва и МО	Россия	27
2	Санкт-Петербург	Россия	15

6. Вывести список абонентов, звонивших только в ночное время.

```
SELECT DISTINCT
    s.phone_number,
    s.full_name,
    COUNT(cl.call_id) as night_calls_count
FROM telecom.subscriber s
JOIN telecom.contract c ON s.phone_number = c.phone_number
LEFT JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
    AND (cl.call_time >= '23:00:00' OR cl.call_time < '07:00:00')
WHERE NOT EXISTS (
    SELECT 1 FROM telecom.call_log cl2
    WHERE cl2.contract_number = c.contract_number
    AND cl2.call_time < '23:00:00'
    AND cl2.call_time >= '07:00:00'
)
GROUP BY s.phone_number, s.full_name
HAVING COUNT(cl.call_id) > 0;
```

Query Query History

```
1 SELECT DISTINCT
2     s.phone_number,
3     s.full_name,
4     COUNT(cl.call_id) as night_calls_count
5 FROM telecom.subscriber s
6 JOIN telecom.contract c ON s.phone_number = c.phone_number
7 LEFT JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
8     AND (cl.call_time >= '23:00:00' OR cl.call_time < '07:00:00')
9 WHERE NOT EXISTS (
10     SELECT 1 FROM telecom.call_log cl2
11     WHERE cl2.contract_number = c.contract_number
12     AND cl2.call_time < '23:00:00'
13     AND cl2.call_time >= '07:00:00'
14 )
15 GROUP BY s.phone_number, s.full_name
```

Data Output Messages Notifications

SQL Sh

	phone_number [PK] character varying (12)	full_name character varying (100)	night_calls_count bigint
1	+79005556677	Ночной Павел Петров...	8

7. Вывести список абонентов, время разговоров которых превышает среднее для этой же зоны.

```
SELECT DISTINCT s.phone_number, s.full_name, z.zone_name
FROM telecom.subscriber s
JOIN telecom.contract c ON s.phone_number = c.phone_number
JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
JOIN telecom.zone z ON cl.zone_code = z.zone_code
WHERE cl.duration > (
    SELECT AVG(cl2.duration)
    FROM telecom.call_log cl2
    WHERE cl2.zone_code = z.zone_code
);
```


Query	Query History
1	SELECT DISTINCT s.phone_number, s.full_name, z.zone_name
2	FROM telecom.subscriber s
3	JOIN telecom.contract c ON s.phone_number = c.phone_number
4	JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
5	JOIN telecom.zone z ON cl.zone_code = z.zone_code
6	WHERE cl.duration > (
7	SELECT AVG(cl2.duration)
8	FROM telecom.call_log cl2
9	WHERE cl2.zone_code = z.zone_code
10	);

Data Output	Messages	Notifications
phone_number	full_name	zone_name
character varying (12)	character varying (100)	character varying (50)
1	+79001112233	Иванов Иван Иванович
2	+79001112233	Иванов Иван Иванович
3	+79001112233	Иванов Иван Иванович
4	+79004445566	Петров Петр Петрович
5	+79004445566	Петров Петр Петрович
6	+79004445566	Петров Петр Петрович
7	+79007778899	Сидорова Анна Сергее...
8	+79007778899	Сидорова Анна Сергее...
9	+79007778899	Сидорова Анна Сергее...

Часть 3:

1. Содержащее сведения обо всех абонентах и их переговорах за прошедший месяц.

```
CREATE OR REPLACE VIEW telecom.v_subscriber_calls_last_month AS
SELECT s.phone_number, s.full_name, s.city, s.status,
       cl.call_id, cl.call_date, cl.call_time, cl.duration,
       z.zone_name, z.country, z.price_per_min,
       (cl.duration * z.price_per_min) as call_cost
FROM telecom.subscriber s
JOIN telecom.contract c ON s.phone_number = c.phone_number
JOIN telecom.call_log cl ON c.contract_number = cl.contract_number
JOIN telecom.zone z ON cl.zone_code = z.zone_code
WHERE cl.call_date >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '1
month')
AND cl.call_date < DATE_TRUNC('month', CURRENT_DATE);
```

-- Проверка

```
SELECT * FROM telecom.v_subscriber_calls_last_month LIMIT 10;
```


Query

Query History

Scratch Pad

1

SELECT * FROM telecom.v_subscriber_calls_last_month LIMIT 10;

Data Output

Messages

Notifications

</

- Найти самую популярную зону звонков за истекший год.

```
CREATE OR REPLACE VIEW telecom.v_most_popular_zone AS
SELECT z.zone_code, z.zone_name, z.country,
       COUNT(cl.call_id) as total_calls,
       SUM(cl.duration) as total_minutes
FROM telecom.call_log cl
JOIN telecom.zone z ON cl.zone_code = z.zone_code
WHERE cl.call_date >= DATE_TRUNC('year', CURRENT_DATE - INTERVAL '1
year')
AND cl.call_date < DATE_TRUNC('year', CURRENT_DATE)
GROUP BY z.zone_code, z.zone_name, z.country
ORDER BY total_calls DESC
LIMIT 1;
```

-- Проверка

```
SELECT * FROM telecom.v_most_popular_zone;
```

Query

Query History

1

-- Проверка

2

SELECT * FROM telecom.v_most_popular_zone;

Data Output

Messages

Notifications

≡

+

SQL

	zone_code integer	zone_name character varying (50)	country character	total_calls bigint	total_minutes bigint
1	1	Москва и МО	Россия	4	1590

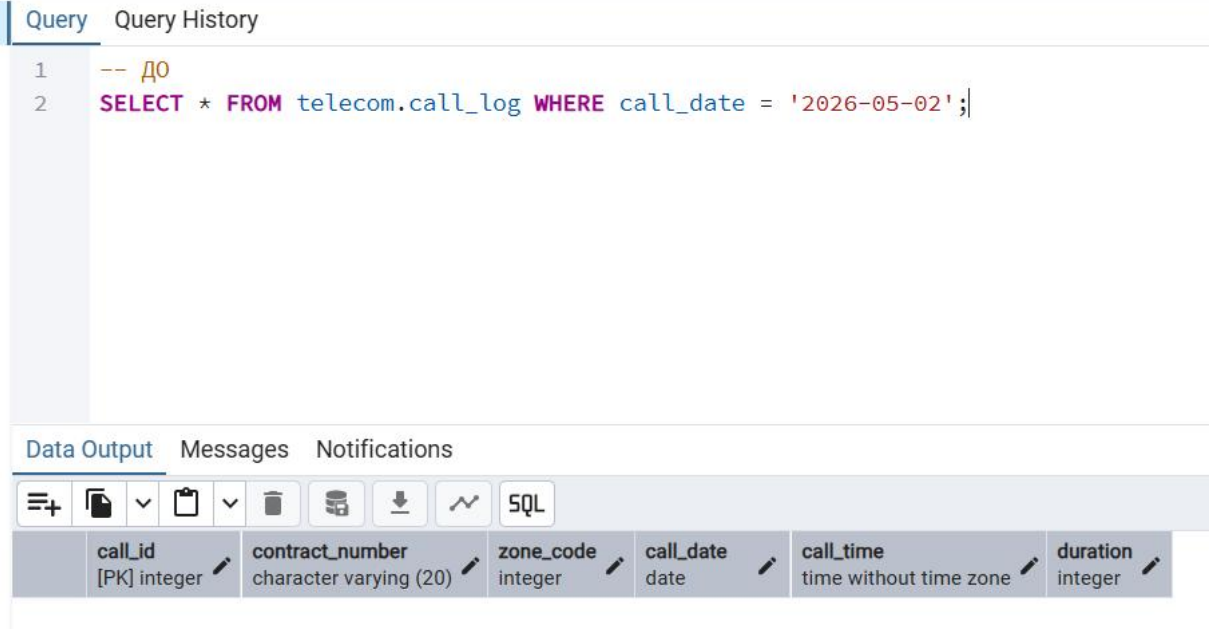
Запросы на модификацию данных

1. INSERT с подзапросом:

Формулировка: Добавить новый звонок на дату 2026-05-02 для всех абонентов с тарифом "Премиум", если звонок на эту дату ещё не был добавлен.

-- ДО

```
SELECT * FROM telecom.call_log WHERE call_date = '2026-05-02';
```



-- INSERT с подзапросом

```
INSERT INTO telecom.call_log (contract_number, zone_code, call_date, call_time, duration)
```

```
SELECT c.contract_number, 1, '2026-05-02', '12:00:00', 60
```

```
FROM telecom.contract c
```

```
WHERE c.tariff_code = (
```

```
    SELECT tariff_code FROM telecom.tariff WHERE name = 'Премиум'
```

```
)
```

```
AND NOT EXISTS (
```

```
    SELECT 1 FROM telecom.call_log cl
```

```
    WHERE cl.contract_number = c.contract_number
```

```
    AND cl.call_date = '2026-05-02'
```

```
);
```

-- ПОСЛЕ

```
SELECT * FROM telecom.call_log WHERE call_date = '2026-05-02';
```

Query

Query History

1

-- ПОСЛЕ

2

SELECT * FROM telecom.call_log WHERE call_date = '2026-05-02';

Data Output

Messages

Notifications

+

≡

</

2. UPDATE с подзапросом:

Формулировка: Увеличить стоимость минуты на 20% в тех зонах, куда совершалось более 2-х звонков за период с 01.04.2026, но только для международных направлений (страна не Россия).

-- Проверка текущих цен в зонах ДО обновления

```

SELECT
    z.zone_code,
    z.zone_name,
    z.country,
    z.price_per_min,
    COUNT(cl.call_id) as calls_count
FROM telecom.zone z
LEFT JOIN telecom.call_log cl ON z.zone_code = cl.zone_code
WHERE cl.call_date >= '2026-04-01'
GROUP BY z.zone_code, z.zone_name, z.country, z.price_per_min
ORDER BY z.zone_code;

```

Query

Query History

1

-- Проверка текущих цен в зонах ДО обновления

2

SELECT

3

z.zone_code,

4

z.zone_name,

5

z.country,

6

z.price_per_min,

7

COUNT(cl.call_id) as calls_count

8

FROM telecom.zone z

9

LEFT JOIN telecom.call_log cl ON z.zone_code = cl.zone_code

10

WHERE cl.call_date >= '2026-04-01'

11

GROUP BY z.zone_code, z.zone_name, z.country, z.price_per_min

12

ORDER BY z.zone_code;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	zone_code [PK] integer	zone_name character varying (50)	country character varying (50)	price_per_min numeric (6,2)	calls_count bigint
1	1	Москва и МО	Россия	1.50	19
2	2	Санкт-Петербург	Россия	1.50	10
3	3	Европа	Германия	15.00	4
4	4	США	США	25.00	3

```

-- UPDATE с подзапросом
UPDATE telecom.zone
SET price_per_min = price_per_min * 1.2
WHERE country != 'Россия'
AND zone_code IN (
    SELECT zone_code
    FROM telecom.call_log
    WHERE call_date >= '2026-04-01'
    GROUP BY zone_code
    HAVING COUNT(call_id) > 2
);

-- ПОСЛЕ
SELECT
    z.zone_code,
    z.zone_name,
    z.country,
    z.price_per_min,
    COUNT(cl.call_id) as calls_count
FROM telecom.zone z
LEFT JOIN telecom.call_log cl ON z.zone_code = cl.zone_code
WHERE cl.call_date >= '2026-04-01'
GROUP BY z.zone_code, z.zone_name, z.country, z.price_per_min
ORDER BY z.zone_code;

```

Query Query History

```

1  -- ПОСЛЕ
2  SELECT
3      z.zone_code,
4      z.zone_name,
5      z.country,
6      z.price_per_min,
7      COUNT(cl.call_id) as calls_count
8  FROM telecom.zone z
9  LEFT JOIN telecom.call_log cl ON z.zone_code = cl.zone_code
10 WHERE cl.call_date >= '2026-04-01'
11 GROUP BY z.zone_code, z.zone_name, z.country, z.price_per_min
12 ORDER BY z.zone_code;

```

Data Output Messages Notifications










SQL

	zone_code [PK] integer	zone_name character varying (50)	country character varying (50)	price_per_min numeric (6,2)	calls_count bigint
1	1	Москва и МО	Россия	1.50	19
2	2	Санкт-Петербург	Россия	1.50	10
3	3	Европа	Германия	18.00	4
4	4	США	США	30.00	3

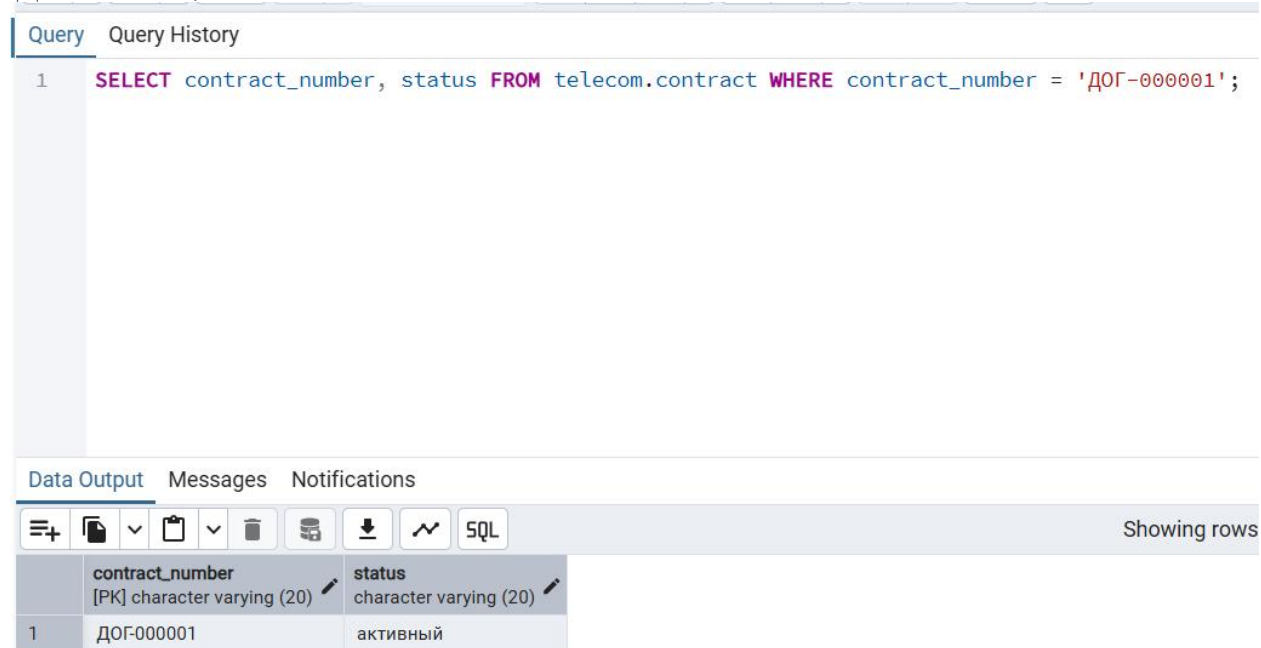
3. DELETE с подзапросом

Формулировка: Удалить звонки абонентов с расторгнутыми договорами.

Для этого нам сначала нужны расторгнутые договоры, сделаем их так:

Проверим договор 1:

```
SELECT contract_number, status FROM telecom.contract WHERE contract_number = 'ДОГ-000001';
```



The screenshot shows a database query interface with a 'Query' tab selected. The query text is: `SELECT contract_number, status FROM telecom.contract WHERE contract_number = 'ДОГ-000001';`. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'contract_number' and 'status'. The table contains one row with the values 'ДОГ-000001' and 'активный'.

	contract_number [PK] character varying (20)	status character varying (20)
1	ДОГ-000001	активный

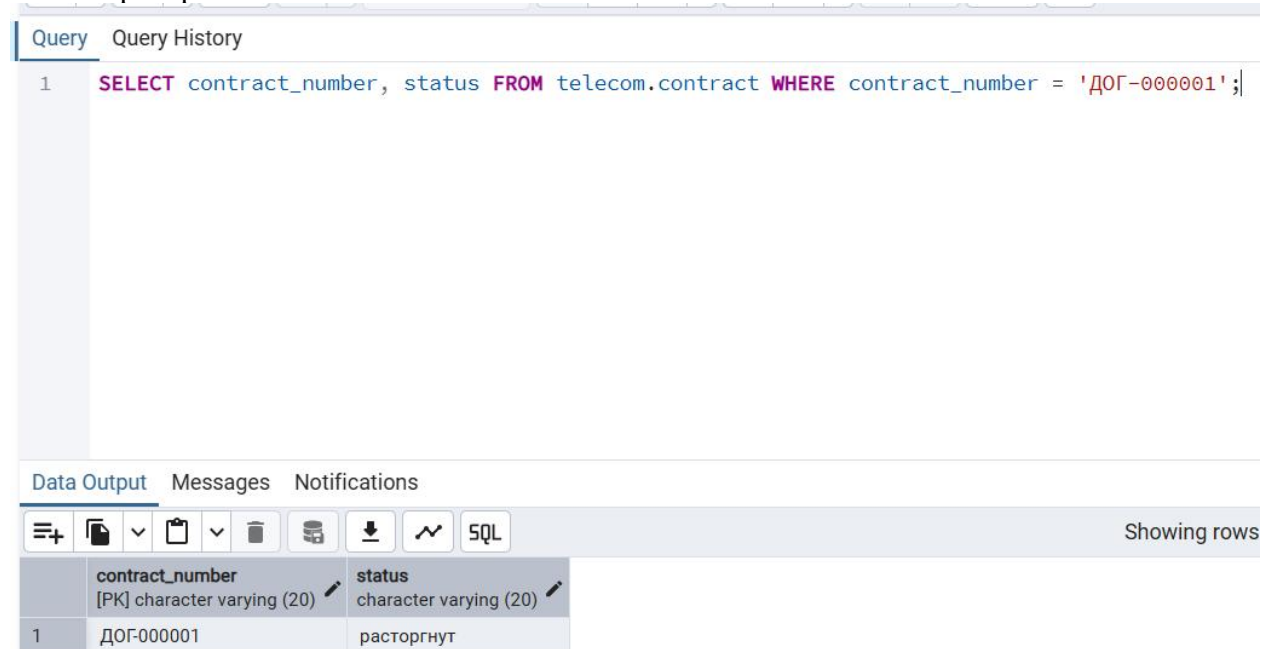
-- Временно установим статус 'расторгнут' для одного договора

```
UPDATE telecom.contract
```

```
SET status = 'расторгнут'
```

```
WHERE contract_number = 'ДОГ-000001';
```

Снова проверим:



The screenshot shows the same database query interface as before, but the status of the contract has been updated. The query text is: `SELECT contract_number, status FROM telecom.contract WHERE contract_number = 'ДОГ-000001';`. The 'Data Output' tab shows the same table structure, but the status is now 'расторгнут'.

	contract_number [PK] character varying (20)	status character varying (20)
1	ДОГ-000001	расторгнут

Далее перейдем к удалению:

-- ДО

```
SELECT cl.call_id, cl.contract_number, cl.call_date, cl.duration  
FROM telecom.call_log cl
```

```
WHERE cl.contract_number = 'ДОГ-000001'
ORDER BY cl.call_id
LIMIT 10;
```

Query

Query History

1

-- ДО

2

SELECT cl.call_id, cl.contract_number, cl.call_date, cl.duration

3

FROM telecom.call_log cl

4

WHERE cl.contract_number = 'ДОГ-000001'

5

ORDER BY cl.call_id

6

LIMIT 10;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	call_id [PK] integer	contract_number character varying (20)	call_date date	duration integer
1	1	ДОГ-000001	2024-01-10	120
2	2	ДОГ-000001	2024-01-11	300
3	47	ДОГ-000001	2026-04-24	450
4	48	ДОГ-000001	2026-04-25	600
5	49	ДОГ-000001	2026-04-26	300
6	54	ДОГ-000001	2026-05-01	150
7	55	ДОГ-000001	2026-04-20	120
8	56	ДОГ-000001	2026-04-21	90
9	57	ДОГ-000001	2026-04-22	180
10	60	ДОГ-000001	2026-04-15	300

```
-- DELETE с подзапросом
DELETE FROM telecom.call_log
WHERE contract_number IN (
    SELECT contract_number
    FROM telecom.contract
    WHERE status = 'расторгнут'
);
```

```
-- ПОСЛЕ
SELECT cl.call_id, cl.contract_number, cl.call_date, cl.duration
FROM telecom.call_log cl
WHERE cl.contract_number = 'ДОГ-000001'
ORDER BY cl.call_id;
```


Query

Query History

1

-- ПОСЛЕ

2

SELECT cl.call_id, cl.contract_number, cl.call_date, cl.duration

3

FROM telecom.call_log cl

4

WHERE cl.contract_number = 'ДОГ-000001'

5

ORDER BY cl.call_id;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

call_id

[PK] integer

contract_number

character varying (20)

call_date

date

duration

integer

После я верну статус активный 1-ому договору:
UPDATE telecom.contract
SET status = 'активный'
WHERE contract_number = 'ДОГ-000001';

Создание индексов

Подготовка:

Для индексов нужно много данных (иначе он ими не пользуется), поэтому я решил сделать много данных с помощью random:

```
INSERT INTO telecom.call_log (contract_number, zone_code, call_date, call_time, duration)
SELECT
```

```
  c.contract_number,
  (random() * 3 + 1)::int,
  date '2026-04-01' + (random() * 30)::int,
  time '09:00:00' + (random() * interval '10 hours'),
  (random() * 600 + 10)::int
```

```
FROM telecom.contract c
```

```
CROSS JOIN generate_series(1, 100);
```

```
ANALYZE telecom.call_log;
```

1. Простой индекс (call_date)

Первый запрос (без индекса):

```
EXPLAIN ANALYZE
```

```
SELECT * FROM telecom.call_log
```

```
WHERE call_date = '2026-04-10';
```

The screenshot shows a database query interface with a 'Query' tab selected. The query entered is:

```
1 EXPLAIN ANALYZE
2 SELECT * FROM telecom.call_log
3 WHERE call_date = '2026-04-10';
```

Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a 'QUERY PLAN' section. The query plan details are as follows:

Step	Operation
1	Seq Scan on call_log (cost=0.00..17.18 rows=15 width=38) (actual time=0.036..0.193 rows=15 loops=...)
2	Filter: (call_date = '2026-04-10'::date)
3	Rows Removed by Filter: 719
4	Planning Time: 1.044 ms
5	Execution Time: 0.207 ms

Создание индексов:

```
CREATE INDEX idx_call_date ON telecom.call_log(call_date);
```

После создания индексов:

Query
Query History

1
2
3

EXPLAIN ANALYZE
SELECT * FROM telecom.call_log
WHERE call_date = '2026-04-10';

Data Output
Messages
Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

QUERY PLAN

text

🔒

1
2
3
4
5
6
7

Bitmap Heap Scan on call_log (cost=4.27..12.45 rows=15 width=38) (actual time=0.038..0.045 rows=...
Recheck Cond: (call_date = '2026-04-10'::date)
Heap Blocks: exact=7
-> Bitmap Index Scan on idx_call_date (cost=0.00..4.26 rows=15 width=0) (actual time=0.033..0.033...
Index Cond: (call_date = '2026-04-10'::date)
Planning Time: 1.251 ms
Execution Time: 0.086 ms

Удаление индексов:

DROP INDEX telecom.idx_call_date;

- Составной индекс (contract_number, call_date)

Первый запрос (без индекса):

EXPLAIN ANALYZE

SELECT * FROM telecom.call_log

WHERE contract_number = 'ДОГ-000001'

AND call_date >= '2026-04-01';

Query
Query History

```

1  EXPLAIN ANALYZE
2  SELECT * FROM telecom.call_log
3  WHERE contract_number = 'ДОГ-000001'
4         AND call_date >= '2026-04-01';

```

Data Output
Messages
Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

QUERY PLAN

text

🔒

1	Seq Scan on call_log (cost=0.00..19.01 rows=99 width=38) (actual time=0.014..0.092 rows=100 loops=...
2	Filter: ((call_date >= '2026-04-01'::date) AND ((contract_number)::text = 'ДОГ-000001'::text))
3	Rows Removed by Filter: 634
4	Planning Time: 0.509 ms
5	Execution Time: 0.107 ms

Создание составных индексов:

```
CREATE INDEX idx_call_contract_date ON telecom.call_log(contract_number, call_date);
```

После создания индексов:

Query
Query History

```

1  EXPLAIN ANALYZE
2  SELECT * FROM telecom.call_log
3  WHERE contract_number = 'ДОГ-000001'
4         AND call_date >= '2026-04-01';

```

Data Output
Messages
Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

QUERY PLAN

text

🔒

1	Bitmap Heap Scan on call_log (cost=5.29..14.77 rows=99 width=38) (actual time=0.074..0.095 rows=...
2	Recheck Cond: (((contract_number)::text = 'ДОГ-000001'::text) AND (call_date >= '2026-04-01'::date))
3	Heap Blocks: exact=8
4	-> Bitmap Index Scan on idx_call_contract_date (cost=0.00..5.27 rows=99 width=0) (actual time=0.0...
5	Index Cond: (((contract_number)::text = 'ДОГ-000001'::text) AND (call_date >= '2026-04-01'::date))
6	Planning Time: 1.211 ms
7	Execution Time: 0.137 ms

Удаление индексов:

```
DROP INDEX telecom.idx_call_contract_date;
```

Выводы

В ходе выполнения лабораторной работы были получены следующие результаты:

1. **Запросы на выборку:** Разработано и выполнено 7 запросов к базе данных TelecomDB согласно варианту 17. Запросы включают агрегатные функции (SUM, COUNT, AVG), фильтрацию по временным интервалам, подзапросы и операторы EXISTS/NOT EXISTS (например, поиск абонентов, звонивших только в ночное время).
2. **Представления (Views):** Созданы 2 представления:
 - v_subscriber_calls_last_month — для просмотра детальной информации о переговорах за прошедший месяц.
 - v_most_popular_zone — для определения самой популярной зоны звонков за истекший год.
3. **Модификация данных:** Реализованы 3 запроса на изменение данных (INSERT, UPDATE, DELETE) с использованием подзапросов. Показано, как можно добавлять, изменять и удалять записи на основе условий из других таблиц (например, повышение стоимости в зонах с большим количеством звонков).
4. **Индексы и производительность:**
 - В таблицу call_log добавлены тестовые данные (700 записей) для корректного тестирования оптимизатора.
 - Проведен анализ планов выполнения запросов с помощью команды EXPLAIN ANALYZE.
 - Простой индекс на поле call_date изменил план выполнения с Seq Scan на Bitmap Heap Scan и ускорил поиск по дате с 0.207 мс до 0.086 мс (в ~2.4 раза).
 - Составной индекс на поля (contract_number, call_date) также был успешно задействован оптимизатором запросов (Bitmap Index Scan), что подтверждает эффективность индексов для сложных условий фильтрации. (Время и не изменилось в лучшую пользу, скорее всего из-за нехватки данных (надо было делать еще больше))